

SINGLE-CYCLE TOY CPU

with a Custom Minimal ISA

Abstract

A single cycle CPU with a custom ISA of 8 instructions
using ZYBO ZYNQ-7000 development board

Pratham Agrawal

Pratham.agrawal.ece@gmail.com

Contents:

Introduction:	2
ISA-Design:	2
Instruction format:	2
Instruction Table:	3
Register file:	3
Some other design choice:	3
Datapath Architecture:	4
Program Counter:	4
Instruction Memory:	4
Decoder:	4
Control Unit:	5
Registers:	5
ALU:	5
Data Memory:	6
Muxes:	6
Branch logic:	6
Design Decisions and Bugs:	6
Test and Simulation:	7
Hardware Implementation:	8
FPGA Board used:	8
Zybo wrapper:	9
XDC file:	9
Synthesis and Implementation Results:	10
Demonstration:	10
Known Limitations:	11
References:	11

Introduction:

This report presents design and implementation of Toy-CPU project assigned as a take away assignment, which consists of 8-instruction ISA, written in Verilog HDL, while the implementation was done on ZYBO ZYNQ-7000 Development Board. This version is an upgraded from earlier independent CPU project, additionally the results were cross checked by comparing pre calculated outputs to the simulated results and further control flow was validated by hardware implementation. This report covers ISA design, datapath structure, some design decisions, simulation results, hardware implementation and limitations of the design.

ISA-Design:

The custom ISA design for this project is designed as per requirement i.e. it consists instructions like {ADD, SUB} arithmetic, {AND, OR} logical, {LOAD, STORE, MOVI} data movement operations and {BEQ} branch operation.

The data word width is fixed at 8-bits, i.e. consistent for ALU, registers, and data memory, thus keeping the design simple and with minimum ISA. Also, the 8-byte memory size is chosen so that it can match the range of 3 bits imm variable, to avoid invalid address generation.

Instruction format:

Instruction set is of 10 bits specifically designed to keep in mind for the bits to be allotted for opcode, destination register, source register and immediate transfer value.

B9	B8	B7	B6	B5	B4	B3	B2	B1	B0
----	----	----	----	----	----	----	----	----	----

Here,

B9-B7: These are the “OPCODE” bits, decided to make a 3-bit opcode so that it stores 8 custom instructions perfectly.

B6-B5: These bits represent the “Destination register” (Rd)

B4-B3: These bits represent the “Source register” (Rs), 2-bit Rd and Rs are chosen so that they can represent four registers used in the design code, as four registers are well suited to test the working of this toy CPU.

B2-B0: These are the bits that stores “Immediate transfer value” (imm), this 3-bit imm gives address range for Load/Store instruction and offset range for BEQ instruction, as the memory used for the purpose is of size 8 bytes.

Instruction Table:

OPCODE	MNEMONICS	SEMANTICS
000	ADD	$Rd = Rd + Rs$
001	SUB	$Rd = Rd - Rs$
010	AND	$Rd = Rd \& Rs$
011	OR	$Rd = Rd Rs$
100	MOVI	$Rd = \text{immediate}$
101	LOAD	$Rd = \text{Mem}[\text{imm}]$
110	STORE	$\text{Mem}[\text{imm}] = Rd$
111	BEQ	If $Rd == 0$: $PC = PC + \text{imm}$

Register file:

4 general purpose registers {R0-R3} are used in ALU operations, the registers are 8 bits wide and can be easily addressed by 2 bits Rd and Rs as operand 1 and operand 2 for calculation.

Initially the registers carry the values as $R0=5$, $R1=10$, $R2=20$, $R3=30$, these values are taken rather than all zeros so as to validate the operations performed, the simulation results are matched with the on paper solved results for the operations.

Some other design choice:

- 1.) BEQ compares Rd with zero rather than any other register arbitrarily because a zero check was sufficient to validate the control flow of the particular instruction.
- 2.) For the instructions which do not use any particular semantic lets say Rs is not used in load and store instructions, then Rs is stored as zero for demonstration. This does not have any effect on the flow, it is done so to maintain a consistency in design so that the reader can easily differentiate.
- 3.) Also, the memory address for data movement instructions were to be adjusted to come from imm field only, because that was sufficient to check and retrieve data from the 8-byte memory.

Datapath Architecture:

Datapath for this Toy-CPU project is consisting following modules:

1. Program Counter
2. Instruction memory
3. Decoder
4. Data memory
5. Control unit
6. General purpose registers
7. ALU

Apart from these modules the Datapath additionally is having two multiplexers supporting the functions related to input and output for ALU, also another logic is added apart from the modules comparing the register value and asserting offset in program counter.

Program Counter:

This module holds current 8-bit instruction address. PC updates its value to PC_next on every clock edge, though it does not know when to increment by '1' and when to jump with the offset, because that is purely decided by top module.

Instruction Memory:

Instruction Memory accepts an 8-bit address and outputs a 10-bit instruction. It works on a case statement taking 8-bit address as argument and then all the 8 ISA instructions are then hardcoded for this toy_cpu with the comments by it specifying what the instruction is doing. A default case is included to ensure a defined instruction output for unspecified addresses.

Also, the instructions are arranged in a order such that they match the results calculated by dry run and the results evaluated by the CPU, the PC will increase with 1 and thus the order would be followed and at last the PC will increment with the offset as the necessary conditions would achieve till the simulation reaches that instruction.

Decoder:

This module is the one that provides opcode and register/memory addresses by slicing the 10-bit instruction into a 3-bit opcode, 2-bit Rd, 2-bit Rs, and 3-bit imm. It does not do any calculations or any other assignment rather than slicing and serving these fields to other modules as per requirement.

Control Unit:

Now this is the module that gives all the opcodes that are provided by decoder module a physical meaning. It maps 5 signals to each opcode i.e. `alu_src`, `reg_write`, `mem_write`, `mem_or_reg`, `branch` using a case statement with opcode as the argument.

1. `alu_src`: It decides whether the input to the alu will be an immediate value or a register defined value. If `alu_src` is 1 then it represents immediate value and 0 represents register input.
2. `reg_write`: If this signal is defined as 1, then the output is to be written back to registers. If any instruction does not require to write back to register then this signal should be marked 0.
3. `mem_write`: This works same as `reg_write`, the only difference is that this signal stores into data memory when asserted HIGH.
4. `mem_or_reg`: If 0 then takes input from registers/ALU and if 1 then input from data memory say LOAD instruction.
5. `branch`: Asserts branch instruction if this signal is 1.

Registers:

4x8 bit registers that stores preloaded values and are used in register-to-register operations, apart from this there are global inputs like clock, reset. Other input signals that assist in deciding whether to read or write from the register, and which register to access out of 4 available and to write what 8-bit data are present, then 2, 8-bits output signals are provided to forward data to other modules.

Reset preloads all the registers to their default values which are intentionally kept as non-zero values, else if the `reg_write` signal is high then the required values can be stored and evaluated.

ALU:

It is purely combinational and performs arithmetic and logical instructions viz ADD, SUB, AND, OR and MOVI. LOAD, STORE and BEQ do not require arithmetic computation by the ALU in the current design. Default case restores the output to all zero. Where the operand 1 of ALU is always destination register the operand 2 can be both source register or immediate value depending on the multiplexer input that we have used to select between the two choices for operand 2.

Data Memory:

This module is very much similar to register module, it has an 8x8 memory array, on resetting it stores zeros at every position, otherwise if the enable is high then the 8-bit data can be stored at the required address.

Muxes:

- ALU input mux: selects between the source register's value and the immediate field, based on `alu_src` needed because `MOVI`'s second operand is an immediate while arithmetic/logic instructions use a register.
- Register write-data mux: selects between the ALU's output and memory's read output, based on `mem_or_reg` needed because `LOAD` writes memory sourced data into a register while every other register-writing instruction writes ALU-computed data.

Branch logic:

A comparison `reg_data1 == 0` checks the destination register's value against zero, this result is AND with the control unit's branch signal which flags `BEQ`-type instructions to produce `branch_taken` ensuring a branch only occurs when both the instruction type and the condition genuinely align. This drives the final PC-next mux, choosing between `pc_addr+1` and `pc_addr+imm`.

Design Decisions and Bugs:

1. **Bug (fixed now):** Register file read port assignment swapped, i.e. `read_reg1` was wired to source register and `read_reg2` with destination register but since everywhere it was calculated with assuming `Rd` as operand one, so the results of subtraction were not coming what it should have come, hence at last fixed by swapping the falsely connected `read_reg` wires again.
2. **Bug (fixed now):** Initially ALU's `MOVI` case was assigned operand 1 instead of operand 2, i.e. the register was getting rewritten with its own value and thus it was corrected after noticing the error in expected output.
3. **Bug (fixed now):** While updating the decoder module to output separate `rs` and `imm` fields, several connecting wires in the top-level module retained widths from the earlier design (e.g., `imm` declared as 8 bits instead of 3 bits, and the instruction wire declared as 8 bits instead of 10 bits after the format change).
4. **Design decision:** The test program is structured so that `MOVI R3,0` executes just before `BEQ` so that it guarantees the assertion of branch change instruction, as when it will compare register value to zero it will result true. This is done only to verify the design functionality.
5. **Design decision:** Rather than resetting all registers to zero, the register file preloads distinct, non-zero values (`R0=5`, `R1=10`, `R2=20`, `R3=30`) on reset.

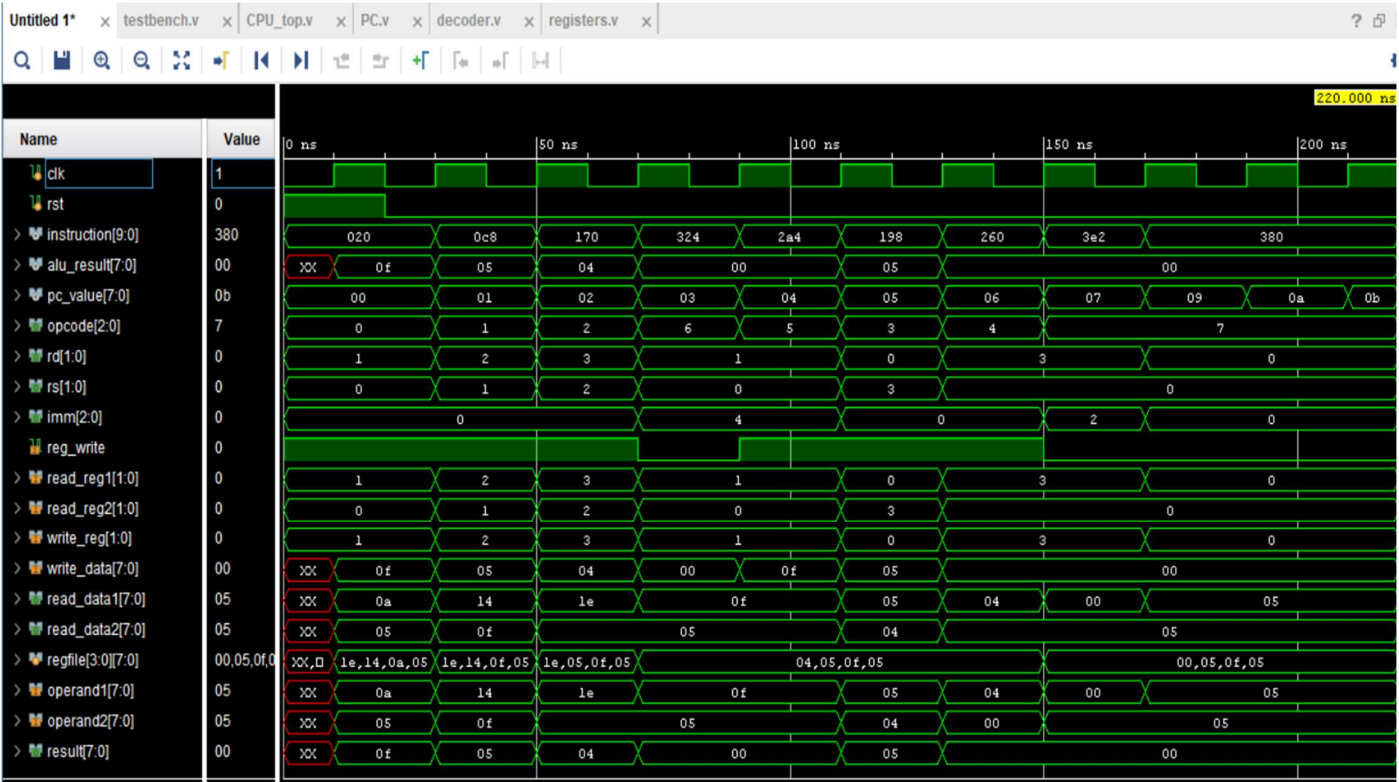
Test and Simulation:

Address	Instruction	Mnemonic	Operation
0	000_01_00_000	ADD	R1=R1+R0
1	001_10_01_000	SUB	R2=R2-R1
2	010_11_10_000	AND	R3=R3&R2
3	110_01_00_100	STORE	Mem[4]=R1
4	101_01_00_100	LOAD	R1=Mem[4]
5	011_00_11_000	OR	R0=R0 R3
6	100_11_00_000	MOVI	R3=0
7	111_11_00_010	BEQ	R3==0, PC=PC+2

Before running simulation, each instruction's effect was manually calculated using the register file's reset values (R0=5, R1=10, R2=20, R3=30). These hand-derived values were then compared against the actual RTL simulation output.

Register	Expected	Simulation output
R0	5	5
R1	15	15
R2	5	5
R3	0	0

All simulated values matched the hand-calculated expectations exactly, ensuring correct behaviour throughout every instruction in the ISA.



Zybo wrapper:

The CPU_top module which was at the highest hierarchy is now instantiated under zybo_wrapper making it the new top module so to interface with physical board I/O, also in this module a slower clock is generated in order to see the changes in outputs through led blinks because the original 125Mhz clock was too fast to be observed properly.

So, a clock divider was implemented using a 27-bit free-running counter, with its most significant bit (counter[26]) used as a divided clock — toggling approximately every 0.5 seconds.

```
reg [26:0] counter = 0;

wire slow_clk;

always @(posedge clk) begin
    counter <= counter + 1;
end

assign slow_clk = counter[26];
```

And so, the clock of CPU_top module is instantiated with this slow_clk signal.

XDC file:

Now this XDC file or constraint file is created to connect physical pins of FPGA board with the ports written in code files, and also to define timing constraints.

```
{    set_property -dict { PACKAGE_PIN L16 IOSTANDARD LVCMOS33 }
[get_ports { clk }];

create_clock -add -name sys_clk_pin -period 8.00 -waveform {0 4} [get_ports { clk }];

set_property -dict { PACKAGE_PIN R18 IOSTANDARD LVCMOS33 } [get_ports { rst
}];

set_property -dict { PACKAGE_PIN M14 IOSTANDARD LVCMOS33 } [get_ports {
led[0] }];

set_property -dict { PACKAGE_PIN M15 IOSTANDARD LVCMOS33 } [get_ports {
led[1] }];

set_property -dict { PACKAGE_PIN G14 IOSTANDARD LVCMOS33 } [get_ports {
led[2] }];

set_property -dict { PACKAGE_PIN D18 IOSTANDARD LVCMOS33 } [get_ports {
led[3] }]; }
```

The above code lines are the XDC file that I have used, this is very minimalistic defines clock and only the 4 LED pins to the lower pins of PC_value field because there are only 4 output leds in Zybo Zynq-7000 development board.

Synthesis and Implementation Results:

1. Utilisation: 101 LUTs, 75 slice registers [CPU core itself uses 100 LUTs / 48 registers; the additional 27 registers and 1 LUT come from the clock divider]
2. memory arrays (instruction memory, data memory) were not mapped to Block RAM due to their small size, and were instead implemented directly in LUTs/registers.
3. WNS: 4.679ns, WHS: 0.324ns; both of them came out to be positive thus ensuring that timing constraints are met.

SYNTHESIZED DESIGN - xc7z010clg400-1 (active)																	
Tcl Console Messages Log Reports Design Runs x Power Timing ? - ⌵ ⌵																	
Q ⌵ ⌵ ⌵ ⌵ ⌵ ⌵ ⌵ ⌵																	
Name	Constraints	Status	WNS	TNS	WHS	THS	TPWS	Total Power	Failed Routes	LUT	FF	BRAMs	URAM	DSP	Start	Elapsed	Run
✓ synth_1	constrs_1	synth_design Complete!								101	75	0.00	0	0	7/14/26 3:51 PM	00:00:19	Vivac
▶ impl_1	constrs_1	Not started															Vivac
✓ impl_2 (active)	constrs_1	write_bitstream Complete!	4.679	0.000	0.324	0.000	0.000	0.100	0	101	75	0.00	0	0	7/14/26 3:52 PM	00:00:57	Vivac

Demonstration:

The lower 4-bits of the 8-bit program counter i.e. pc_value[3:0] are displayed on the board's 4 user LEDs, due to the limitations of board's display option this was the most suited display option to demonstrate the working of the code.

1. When reset i.e. on holding the R18 button on the board, the registers are loaded with the pre-loaded values instead of zero as discussed above in details, and PC counter goes to zero thus all 4 LEDs turns off until R18 is released.
2. On release of reset button, PC counter can be seen incrementing every 0.5 seconds.
3. After displaying 7 the counter jumps to 9 instead of 8 validating the working of BEQ instruction properly.
4. Beyond address 7, execution enters the default no-op case, causing PC to continue incrementing harmlessly; since only 4 bits are displayed, the visible count wraps every 16 real PC increments.
5. Due to the board's limited 4-LED output and time constraints, only program counter and control-flow were directly demonstrated on physical hardware. Arithmetic, logic, and memory operation correctness were verified through RTL simulation, however, the same synthesized design running correctly on real hardware, including successful branch execution, provides strong indirect confidence in the correctness of these operations as well.

*Video reference is added with the mail for demonstration.

Known Limitations:

1. Limited immediate value range: The 3-bit imm field restricts immediate values from 0-7, and thus memory address for memory based instruction to 0-7 also. This trade off was done to keep the instruction field 10-bit, which was sufficient in order to store 3-bit opcode and 2-bit register address.
2. BEQ checks only that the destination register is equal to zero or not rather than comparing two arbitrary registers.
3. As for now the increment of PC address was not made to loop after a wrap , it only goes forward however it can be implemented further with some design thinking.
4. There is no interrupt handling, and since this is a single cycle CPU thus no pipeline hazards are implemented.
5. Every instruction completes in exactly one clock cycle regardless of complexity, which is simple and predictable but not performance-optimized compared to pipelined or multi-cycle architectures.

References:

1. Digilent Inc., "Zybo Reference Manual," available at:
<https://digilent.com/reference/programmable-logic/zybo/reference-manual>.
2. National Tsing Hua University, CS4100 Course Notes, "Single-Cycle Processor Design," available at: <https://www.cs.nthu.edu.tw/~king/courses/cs4100/P51-proc.pdf>
3. YouTube video series on single-cycle MIPS architecture and Verilog implementation:
<https://www.youtube.com/watch?v=HFfSKQHnRI&list=PLqwDtnvV0VNxNxENG0s8MmWjzvH1ICSPD&index=3>